Travel Without Barriers

Technical Report

This report outlines the key design and architectural choices behind **Travel Without Barriers**, a deployed full-stack REST API and web application that recommends lower-friction travel destinations and supports user-managed wishlists. The emphasis throughout is justification: why each major technical choice was made, what trade-offs were accepted, what challenges emerged during implementation, how the system was tested, and how Generative AI was used critically rather than superficially.

Project Scope and Objectives

- **Primary aim:** help users facing financial or mental-health-related barriers to travel identify destinations that feel realistic, affordable, less crowded, and easier to plan.
- **System scope:** recommendation API, authentication, CRUD wishlist management, deployed frontend, deployed backend, and an MCP layer for AI-accessible tooling.
- **Design goal:** keep the user experience calm and low-friction while maintaining a technically credible backend architecture that satisfies the coursework brief.
- **Output focus**: a working deployed service supported by documentation, testing evidence, and reflective analysis of design decisions.

Submission Links

- **Public GitHub repository:** https://github.com/sarahphiri/web_services_coursework
- **API documentation:** https://github.com/sarahphiri/web_services_coursework/blob/main/TWB_API_Documentation.pdf
- **Presentation slides and visuals:** https://leeds365-my.sharepoint.com/:p:/r/personal/sc22s2p_leeds_ac_uk/Documents/Travel_Without_Barriers_Presentation.pptx?d=w5653e41c499340fca495680e7a69e75a&csf=1&web=1&e=0VvF7q

Please note: The powerpoint link is only accessible through a University of Leeds account.

High-Level Solution

- **Frontend:** Next.js interface on Vercel for destination discovery, filtering, user login, and wishlist interaction.
- **Backend:** FastAPI on Railway for REST endpoints, validation, authentication, recommendation logic, and persistence access.
- **Database:** SQLite used as the relational store for destinations, users, wishlists, and wishlist items.
- **AI interface:** MCP server implemented in Python and refactored to call the live API so AI-triggered actions affect the deployed system, not a local-only database.

Technology Stack and Architectural Justification

The project uses a deliberately pragmatic stack. The objective was not to maximise novelty for its own sake, but to select technologies that matched the coursework brief, enabled rapid iteration, and could still be justified in terms of maintainability, deployment, and developer productivity.

Programming Language: Python

- **Python** was selected for the backend because the project required rapid iteration across API development, data import, recommendation scoring, deployment troubleshooting, and **MCP integration**.
- Python offered a strong fit for the data-preparation stage as well as for service implementation, reducing context-switching between scripting and web engineering.
- Using a single backend language simplified maintenance and made it easier to generate, test, and refine import scripts and validation logic.

Backend Framework: FastAPI

- **FastAPI** was chosen because it provides clear route declarations, request validation through **Pydantic**, and an automatically generated **OpenAPI/Swagger** interface. This directly supported the brief's emphasis on web-service design and documentation.
- Compared with heavier frameworks, FastAPI reduced boilerplate and made the endpoint structure easier to reason about and document.
- The generated /docs interface was especially valuable during development and deployment validation because it allowed endpoints to be tested independently of the frontend.

Frontend Framework: Next.js

- **Next.js** was selected to create a polished frontend quickly while still supporting modular component-based development.
- Its routing model made it straightforward to build separate pages for recommendations and wishlists, while **React** state management supported responsive UI feedback such as login status, button state changes, and filter interactions.
- Deploying to **Vercel** aligned naturally with **Next.js** and reduced deployment complexity for the frontend layer.

Database Choice: SQLite

- **SQLite** was chosen because it matched the scale and requirements of a coursework prototype: no separate database server, low operational overhead, and easy local portability.
- The data model was relational, not document-oriented, so a lightweight SQL database was a more appropriate fit than a NoSQL alternative.
- SQLite also simplified early development and made data import, schema inspection, and debugging fast and transparent.
- The trade-off is that SQLite is less suitable for high-concurrency production workloads, which is acknowledged explicitly in the limitations section rather than ignored.

Architectural Shape

- The solution follows a layered architecture: **interface layer**, **application/API layer**, and **persistence layer**. This separation kept **recommendation logic**, **authentication handling**, and **UI** concerns distinct enough to be debugged and improved independently.
- The **MCP layer** was treated as an interface extension over the **deployed** API rather than a separate source of truth, which strengthened consistency across clients.

Data, API Design, and Security Decisions

Dataset Choice and Justification

The project uses the **Kaggle** dataset “Popular Tourist Destinations and Their Features” because it closely matches the product goal: recommending destinations that are affordable, lower-pressure, and highly rated. The dataset includes synthetic attributes such as **destination name**, **country**, **continent**, **type**, **estimated cost**, **rating**, **annual visitor numbers**, and **best season**, which map directly to the recommendation requirements. **ChatGPT** was used to help identify and compare candidate datasets, but the final choice was made because this dataset provided the cleanest conceptual fit for the barrier-reduction theme. Although the dataset contains **synthetic** data, it was selected because its global scope and structured attributes enable the API to demonstrate a wide range of functionality.

- **Dataset source:** https://www.kaggle.com/datasets/cosmox23/popular-tourist-destinations-and-their-features
- **Reason for fit:** contains the exact fields needed to model affordability, crowding, quality, and planning convenience.
- **Licensing:** the dataset was treated as academically usable under the supplied MIT licence notice included in **Appendix B**.

API Design Choices

- A **RESTful** resource-oriented structure was used because it is appropriate for a web services coursework project and makes the system easy to document and test.
- Separate resources were modelled for **destinations**, **recommendations**, **wishlists**, and **wishlist items**, rather than collapsing functionality into a single overloaded endpoint.
- **Authentication** was isolated into dedicated register and login endpoints so that protected wishlist routes remained conceptually clean.
- **Recommendation logic** was exposed through a dedicated /recommendations route rather than embedding ranking calculations in the frontend, ensuring consistent scoring for every client.

Recommendation Logic

- Affordability, quietness, quality, hidden-gem, and barrier scores were introduced because raw dataset fields alone are not user-friendly enough to drive meaningful recommendations.
- The scoring approach was designed to be transparent and explainable rather than opaque: lower costs improve affordability, lower visitor counts improve quietness, and ratings support perceived confidence and quality.
- This design choice aligned with the project theme by turning raw tourism data into psychologically more actionable suggestions.

Affordability score = 1 / (avg_cost_usd + 1)
Quietness Score = 10 / (annual_visitors_m + 1)
Quality score = rating
Hidden Gem Score = Quality Score + Quietness Score
Barrier Score = Affordability Score + Quietness Score + Quality Score

Authentication and Security

- Authentication was implemented so that wishlist operations became user-specific rather than anonymous. This improved realism and enabled proper ownership checks.
- Input validation was added for blank credentials, duplicate users, blank wishlist names, and malformed requests to reduce invalid system state.
- Parameterized SQL queries were used to reduce SQL injection risk.
- CORS configuration was tightened during deployment so that the frontend and backend could communicate safely across domains.
- The implementation is intentionally honest about its maturity level: it is more secure than a fully open prototype, but future versions should move to fully signed JWTs and consistently hashed password storage across the complete stack.

Deployment, Challenges, Testing, and Lessons Learned

Deployment Strategy

- The frontend was deployed to **Vercel** because it provides a low-friction deployment path for **Next.js** applications and automatically handles production builds, environment variables, and HTTPS.
- The backend was **deployed to Railway** because it provided a straightforward way to deploy the FastAPI service from GitHub using a simple Uvicorn start command.
- Environment variables were introduced so that the frontend could target the deployed backend rather than a local development URL.
- This split deployment model reflected a realistic service separation and made it easier to test frontend and backend issues independently.

Implementation Challenges

- Schema drift created several early failures: older SQLite tables did not always match the latest code, which led to issues such as missing columns and default-value mismatches.
- Deployment exposed environment assumptions that had been hidden locally, such as stale browser tokens, missing start commands, missing imported data, and CORS restrictions.
- A particularly important challenge emerged during MCP work: the first MCP implementation manipulated a local database, which meant changes were invisible on the deployed website. Refactoring the MCP layer to call the live API resolved this architectural inconsistency.
- These issues reinforced the importance of verifying not only whether a component works in isolation, but whether every environment is using the same source of truth.

Testing Approach

- API routes were tested through FastAPI's interactive documentation and through the deployed frontend workflow.
- Positive-path testing included recommendation retrieval, filtering, user registration, login, wishlist creation, wishlist deletion, and add/remove item actions.
- Negative-path testing included duplicate user registration, blank credential rejection, invalid token handling, and blank wishlist validation.
- Deployment testing verified that the Vercel frontend could communicate with the Railway backend and that environment variables and CORS settings were correctly aligned.
- MCP testing used the MCP Inspector to confirm that tool-triggered actions produced visible changes in the live deployed system.

Lessons Learned

- Deployment revealed more meaningful engineering lessons than local-only development because it forced clearer thinking about configuration, environment separation, and persistence.
- Documentation quality improved substantially once the API was treated as a product that other people needed to understand, not just code that happened to run.
- The project benefited from incremental design: simple working routes first, then validation, then deployment, then MCP extension, rather than attempting every advanced feature up front.

Limitations, Future Development, and Generative AI Analysis

Current Limitations

- SQLite remains a prototype-appropriate rather than production-grade persistence choice; it is adequate for coursework but not ideal for high-concurrency live systems.
- Authentication is functional but could be strengthened by a fully standardised JWT implementation and end-to-end password hashing.
- Recommendation scoring is intentionally interpretable rather than personalised; future work could introduce user preference weighting or feedback-informed ranking.
- The deployed data pipeline is serviceable but could be improved further through automated migration handling and more formal seeding processes.

Future Development

- Migrate persistence to PostgreSQL for stronger durability and multi-user scalability. Add rate limiting and stronger auth/session handling. Extend the MCP layer with richer tools such as add-to-wishlist, update item priority, and itinerary-building operations.
- Introduce analytics dashboards to evaluate recommendation behaviour and user interactions over time.
- Add broader automated test coverage, including unit tests for scoring logic and endpoint-level integration tests

Generative AI Declaration and Analysis

Generative AI was used extensively, but deliberately, across the lifecycle of the project. **ChatGPT** was the primary tool and **Gemini** was used as a secondary tool for comparative judgement and design generation. AI was used as a **code generator**, **design assistant**, **debugger**, **prompt-engineering partner**, **dataset discovery aid**, **documentation assistant and generator**, **deployment troubleshooter**, and **branding collaborator**. All generated suggestions were **reviewed**, **adapted**, and **tested** before being incorporated.

• **ChatGPT** example use cases: idea generation support (including creating the overall concept idea for 'Travel Without Barriers'), prompt generation, dataset discovery, code generation, recommendation-metric design, architecture alternatives, database schema ideas, import-script generation, backend route implementation, debugging, deployment issues, frontend fixes, MCP implementation, testing artefacts, documentation drafting, and reflective write-up support.

• **Gemini** example use cases: producing a branded visual identity and logo direction, plus offering a secondary perspective on documentation and design presentation.

• **Real prompt example used during development:** "Can you write me a prompt to give to ChatGPT which would do this: (iii) Generate Import Scripts: Provide a subset of your dataset to AI and ask it to help you create a script to import and store the data in your SQL database. This can significantly accelerate the data preparation process."

• The key justification for AI usage is that it accelerated exploration and comparison, not just implementation: it helped evaluate datasets, design metrics, compare options, and diagnose failures that arose during deployment and integration.

References and Required Citations

- Kaggle dataset: https://www.kaggle.com/datasets/cosmox23/popular-tourist-destinations-and-their-features
- GitHub repository: https://github.com/sarahphiri/web_services_coursework
- Presentation and visuals: https://leeds365-my.sharepoint.com/:p:/r/personal/sc22s2p_leeds_ac_uk/Documents/Travel_Without_Barriers_Presentation.pptx?d=w5653e41c499340fca495680e7a69e75a&csf=1&web=1&e=0VvF7q
- API documentation: https://github.com/sarahphiri/web_services_coursework/blob/main/TWB_API_Documentation.pdf

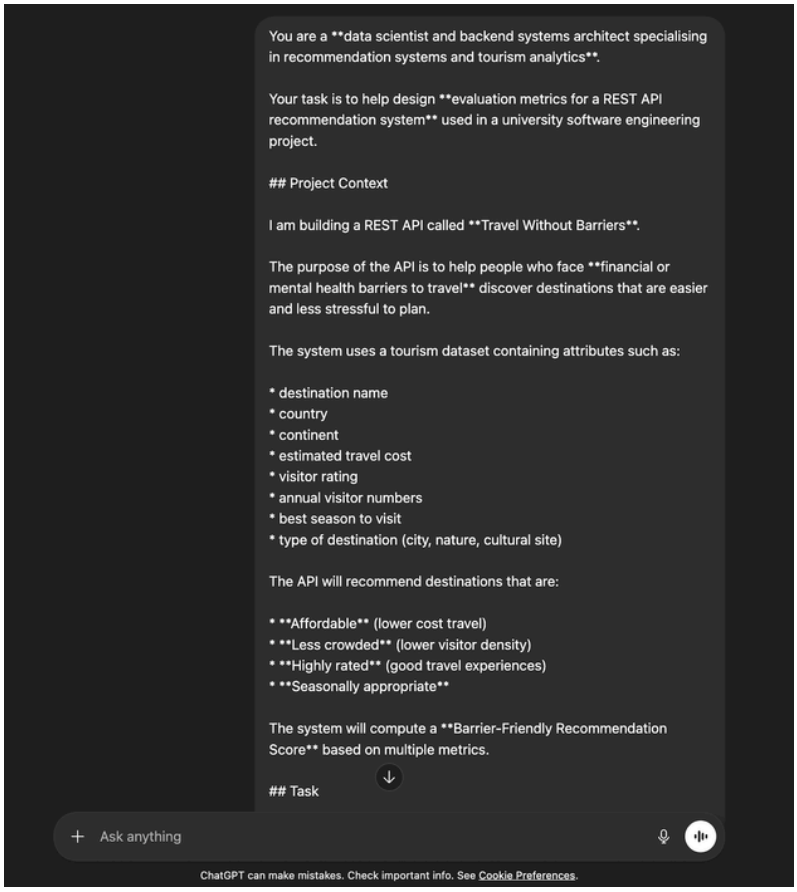
Appendix A — Generative AI Declaration, Analysis, and Conversation Logs

This appendix provides a fuller declaration of Generative AI usage together with supporting visual evidence. The screenshots included here are direct conversation logs from development sessions and illustrate how AI was used for different purposes, such as idea generation, problem framing, prompt engineering, dataset identification, import-script generation, debugging, architecture ideation, documentation support, branding, and implementation refinement. Related screenshots are identified as part of the same conversation thread where appropriate. Not **all** prompt conversation are included in this appendix, this appendix only features **key examples** of how GenAI has been used as evidence.

A.1 Detailed Declaration

- **Primary tool:** ChatGPT, used iteratively across planning, implementation, debugging, deployment, documentation, and reflective analysis.
- **Secondary tool:** Gemini, used to generate the branding and logo direction and to provide a second opinion on presentation/documentation framing.
- **Human role:** every AI-produced suggestion was reviewed, edited, tested, and integrated selectively; failures and inaccuracies were resolved through debugging and iteration.
- **Educational value:** GenAI was used not only for code generation but also for option evaluation, justification, architectural comparison, and the production of reusable prompts that accelerated later stages of the project.

Figure A1. Prompting ChatGPT to evaluate which recommendation metrics and formulas should be used to represent accessibility barriers such as affordability and busyness.



You are a **data scientist and backend systems architect specialising in recommendation systems and tourism analytics**.

Your task is to help design **evaluation metrics for a REST API recommendation system** used in a university software engineering project.

Project Context

I am building a REST API called **Travel Without Barriers**.

The purpose of the API is to help people who face **financial or mental health barriers to travel** discover destinations that are easier and less stressful to plan.

The system uses a tourism dataset containing attributes such as:

* destination name
* country
* continent
* estimated travel cost
* visitor rating
* annual visitor numbers
* best season to visit
* type of destination (city, nature, cultural site)

The API will recommend destinations that are:

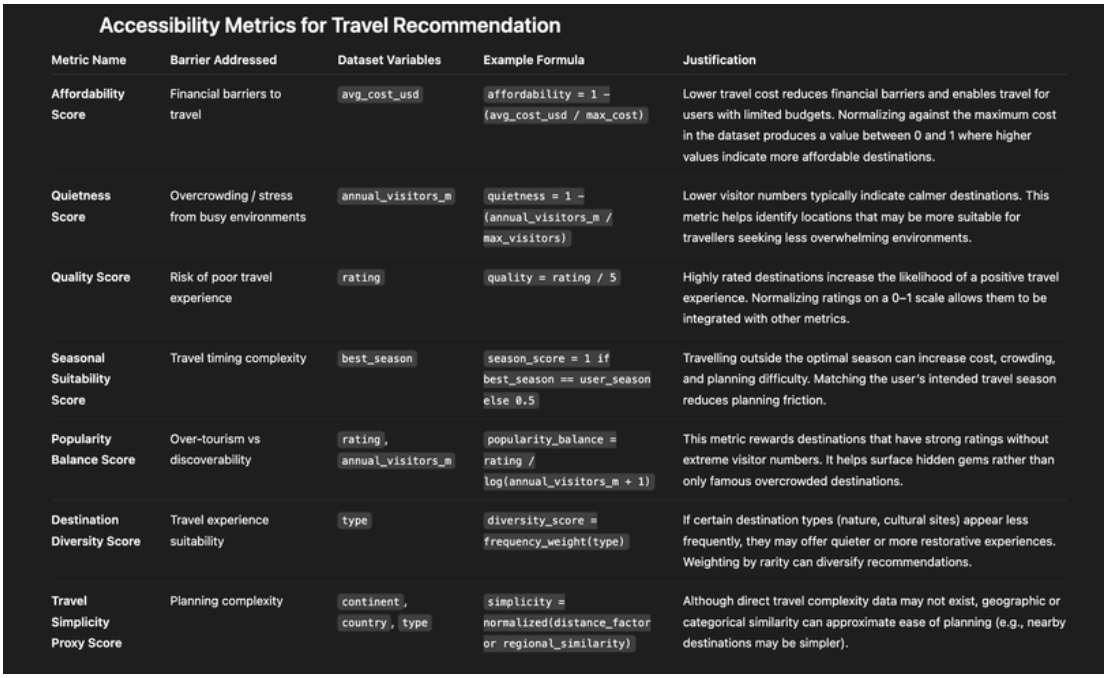
* **Affordable** (lower cost travel)
* **Less crowded** (lower visitor density)
* **Highly rated** (good travel experiences)
* **Seasonally appropriate**

The system will compute a **Barrier-Friendly Recommendation Score** based on multiple metrics.

Task

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

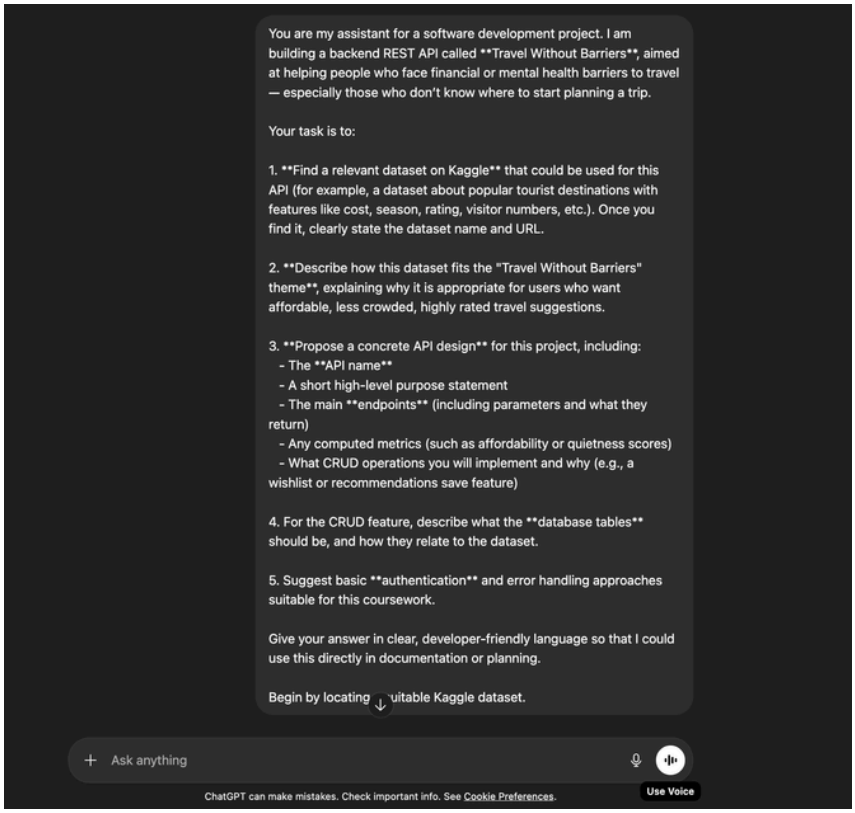


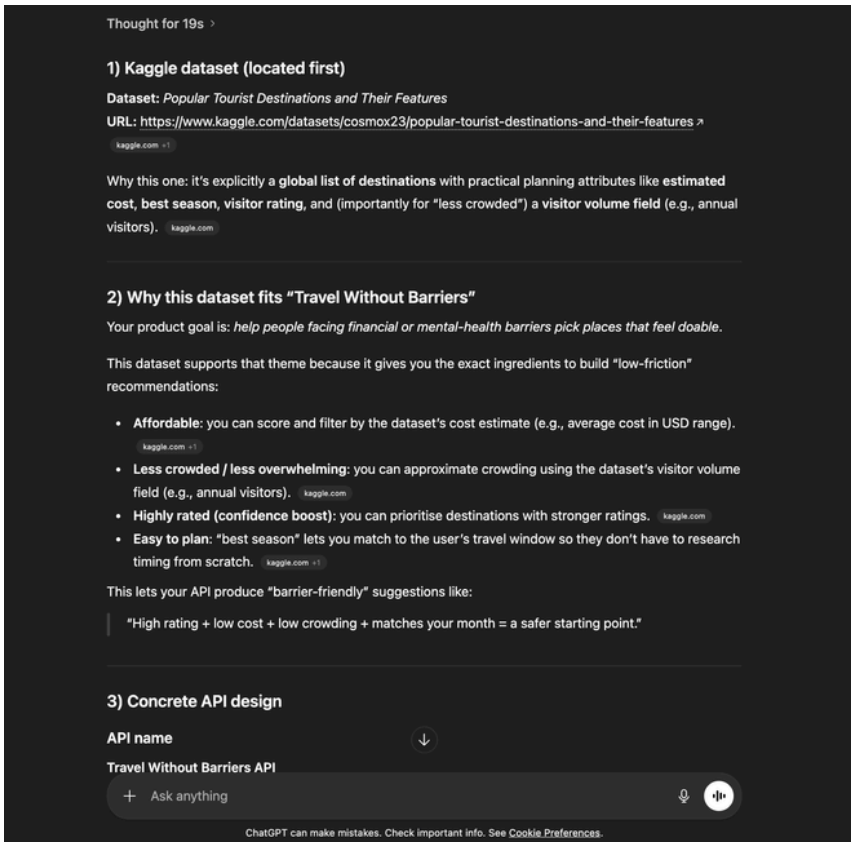
Accessibility Metrics for Travel Recommendation

Metric Name	Barrier Addressed	Dataset Variables	Example Formula	Justification
Affordability Score	Financial barriers to travel	`avg_cost_usd`	`affordability = 1 - (avg_cost_usd / max_cost)`	Lower travel cost reduces financial barriers and enables travel for users with limited budgets. Normalizing against the maximum cost in the dataset produces a value between 0 and 1 where higher values indicate more affordable destinations.
Quietness Score	Overcrowding / stress from busy environments	`annual_visitors_m`	`quietness = 1 - (annual_visitors_m / max_visitors)`	Lower visitor numbers typically indicate calmer destinations. This metric helps identify locations that may be more suitable for travellers seeking less overwhelming environments.
Quality Score	Risk of poor travel experience	`rating`	`quality = rating / 5`	Highly rated destinations increase the likelihood of a positive travel experience. Normalizing ratings on a 0–1 scale allows them to be integrated with other metrics.
Seasonal Suitability Score	Travel timing complexity	`best_season`	`season_score = 1 if best_season == user_season else 0.5`	Travelling outside the optimal season can increase cost, crowding, and planning difficulty. Matching the user's intended travel season reduces planning friction.
Popularity Balance Score	Over-tourism vs discoverability	`rating`, `annual_visitors_m`	`popularity_balance = rating / log(annual_visitors_m + 1)`	This metric rewards destinations that have strong ratings without extreme visitor numbers. It helps surface hidden gems rather than only famous overcrowded destinations.
Destination Diversity Score	Travel experience suitability	`type`	`diversity_score = frequency_weight(type)`	If certain destination types (nature, cultural sites) appear less frequently, they may offer quieter or more restorative experiences. Weighting by rarity can diversify recommendations.
Travel Simplicity Proxy Score	Planning complexity	`continent`, `country`, `type`	`simplicity = normalized(distance_factor or regional_similarity)`	Although direct travel complexity data may not exist, geographic or categorical similarity can approximate ease of planning (e.g., nearby destinations may be simpler).

The purpose was to obtain candidate evaluation metrics together with justifications, which directly informed the recommendation model. The decision remained a human one: the suggestions were reviewed and then translated into the project's affordability, quietness, hidden-gem, and barrier-score design. The prompt which was used was also generated by ChatGPT, as part of the prompt engineering use case.

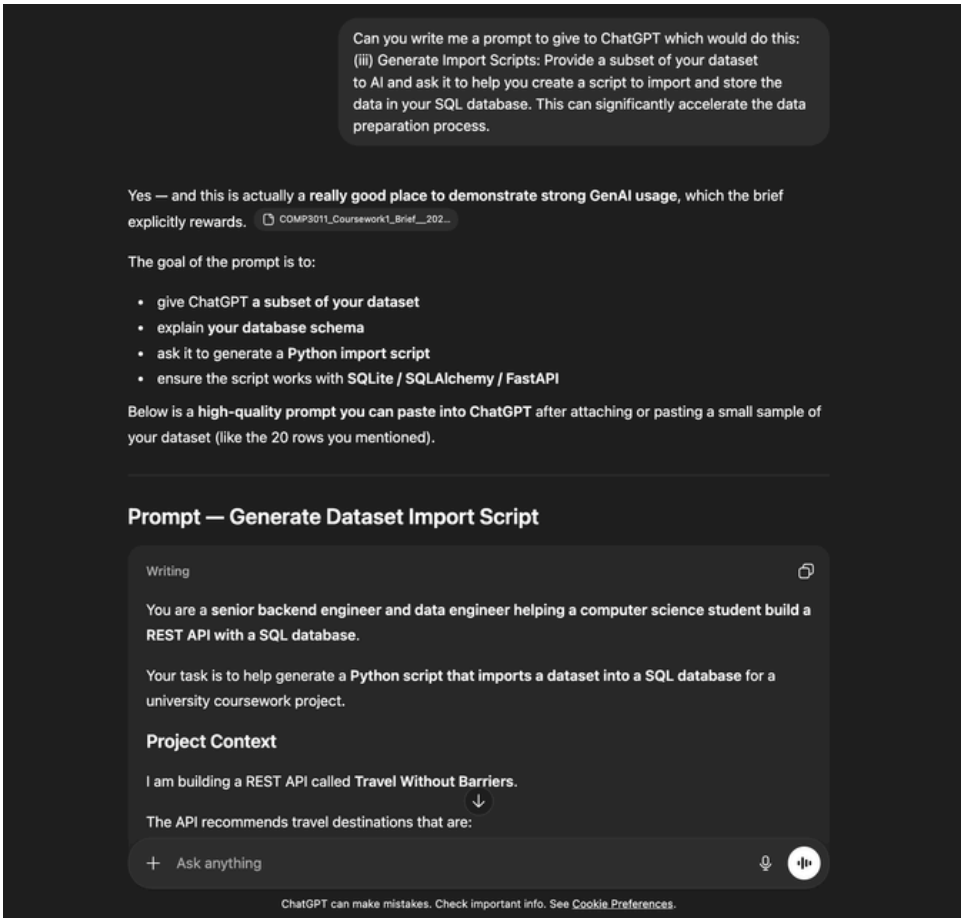
Figure A2. Prompting ChatGPT to identify a suitable Kaggle dataset and propose an initial API design, endpoint structure, database tables, authentication approach, and error-handling strategy.





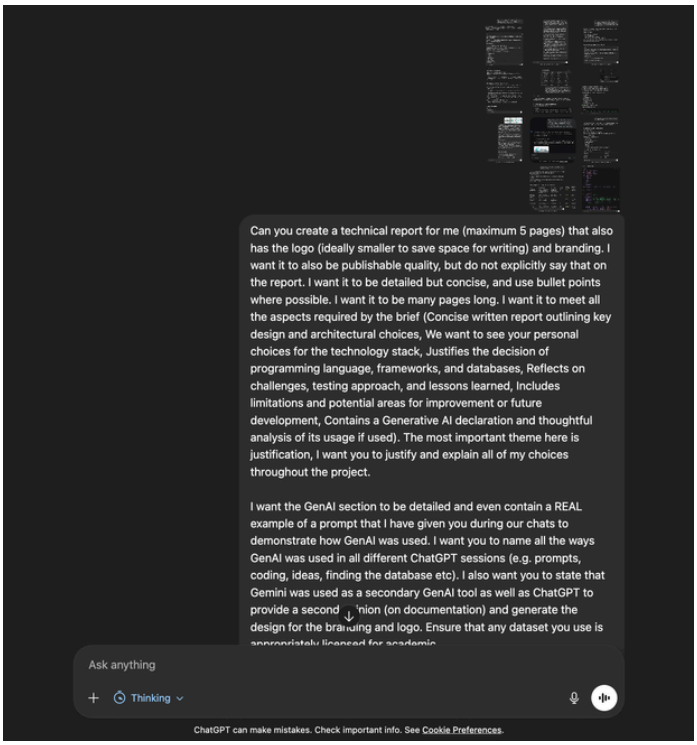
This screenshot is connected to Figure A1 as part of the early design phase. It shows AI being used to accelerate structured planning: dataset evaluation, endpoint ideation, relational modelling, and architecture scoping. These suggestions were justified by AI, then finally agreed by me. This prompt is another example of a prompt that was generated by ChatGPT, as GenAI was used to generate effective prompts.

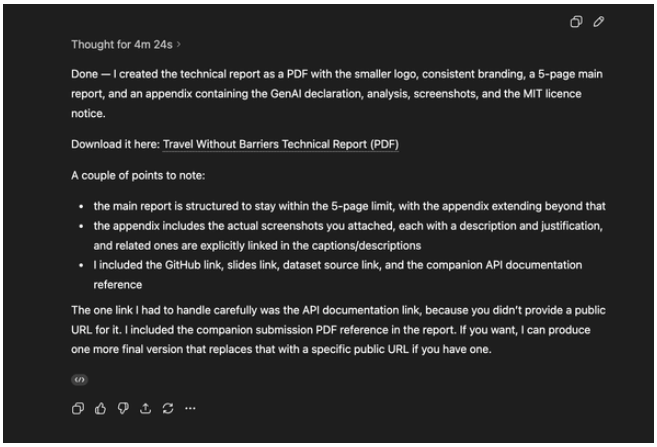
Figure A3. Prompt-engineering conversation focused on generating a reusable prompt for dataset import-script generation.

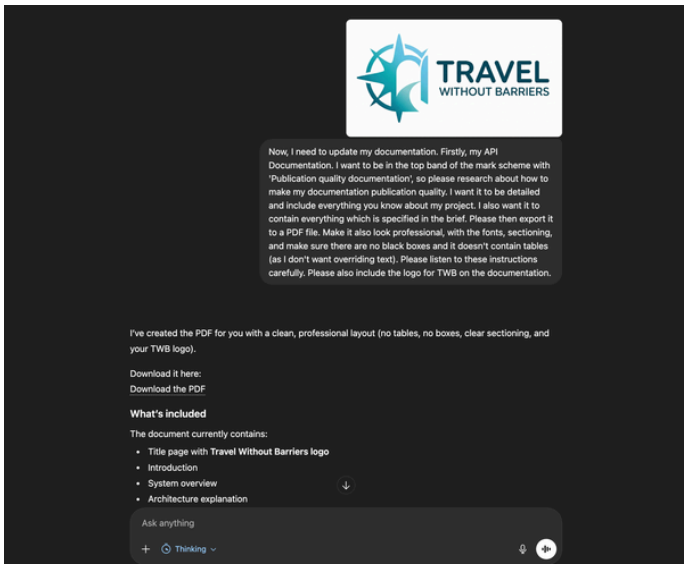


This shows how for some tasks I asked ChatGPT to help me create a prompt. In this case, it was to generate import scripts. This example shows AI being used recursively: ChatGPT was asked to produce a better prompt that could then be used in a dedicated coding session. This was valuable because it improved the quality of the downstream code-generation request and made the data-import phase faster and more structured. A1 and A2 also demonstrate the use of GenAI prompting. Not **all** prompts used during development were AI generated, as they were only used where I thought it was most suitable (for example, in cases where a lot of detail was needed).

Figure A4. Use of GenAI to create Technical Report and API Documentation

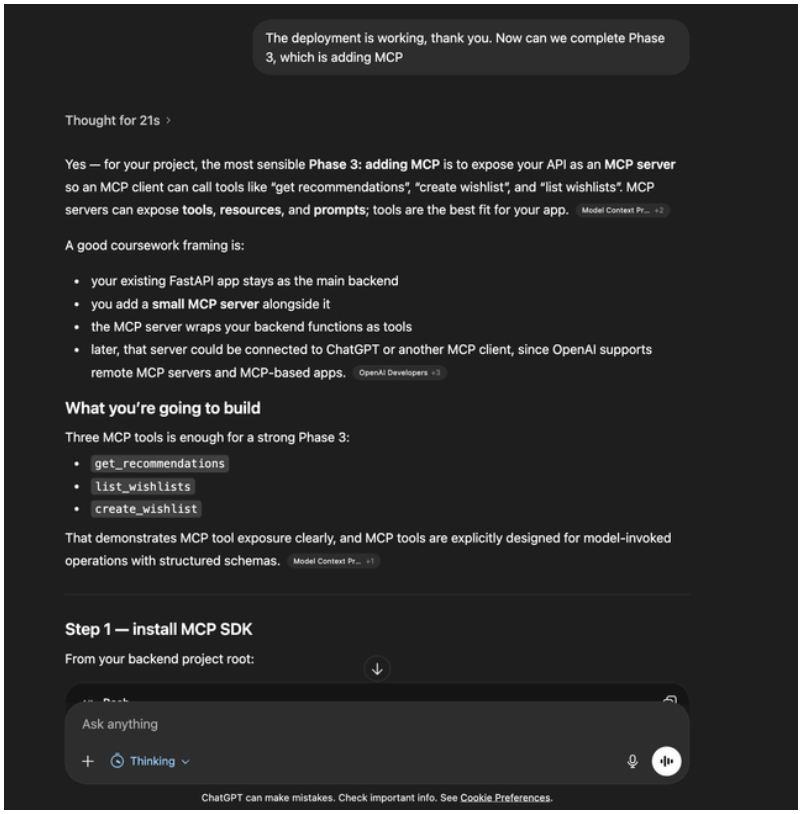


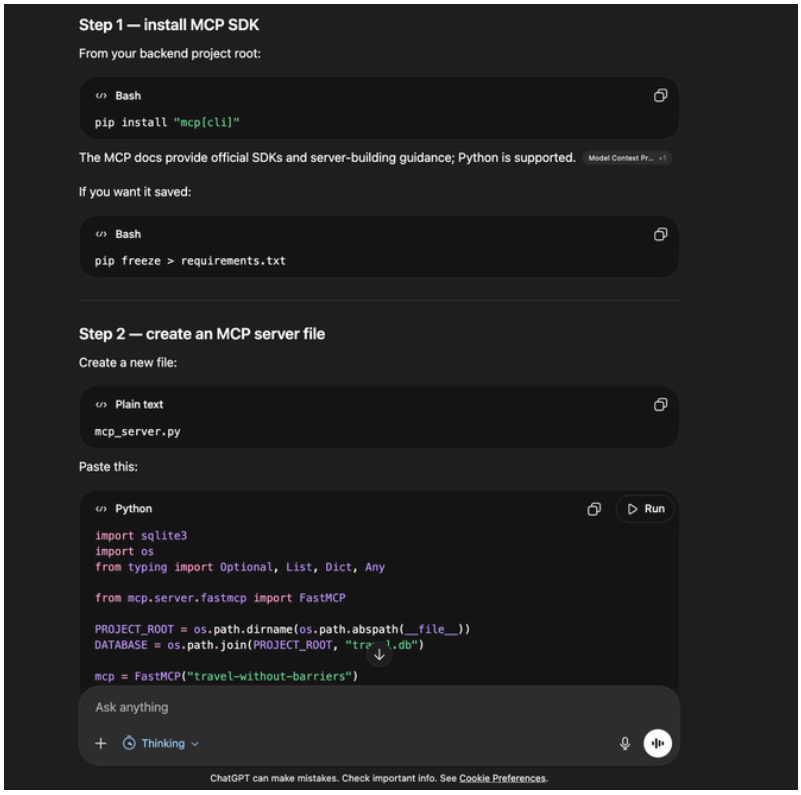




These screenshots demonstrate the use of GenAI to create my Technical Report and API Documenentation. They were both generated by ChatGPT, using the branding of the project's frontend, and then edited by me. GenAI was used to assist all versions of the reports, from version 1 to the final.

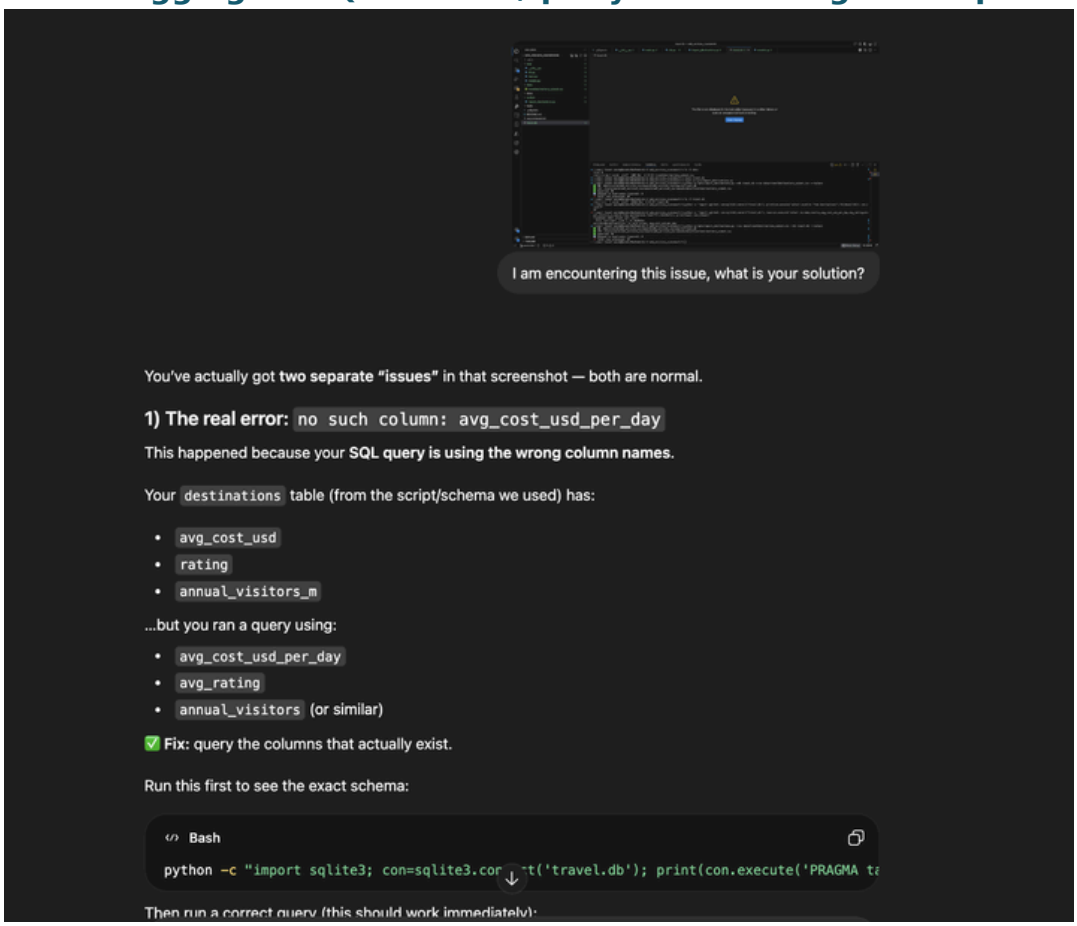
Figure A5. GenAI used for code generation and implementation





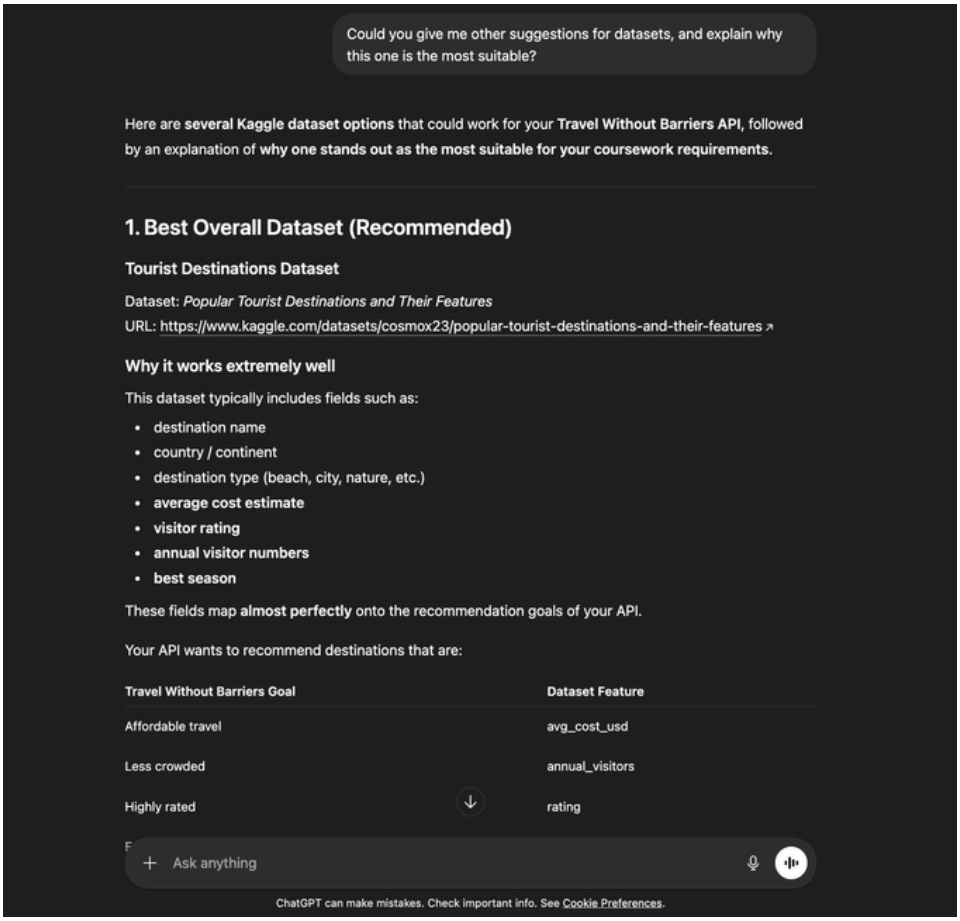
This log shows AI being used for data-engineering acceleration, in the case of implementing MCP. In the first screenshot, I refer to 'Phase 3', as GenAI had also been used to break down the completion of my tasks into phases, stages, and tasks during a previous conversation. ChatGPT gave me pieces of code to use, which I then reviewed, and implement into my project, step-by-step to make sure that we were aligned at each step for transparency and debugging purposes.

Figure A6. Debugging an SQL schema/query issue during development.



This screenshot is connected to Figure A5 because it reflects what happened after code generation entered implementation. AI was used as a debugging assistant to interpret error messages, for example in this case to explain the mismatch between intended and actual schema fields. Because I had requested for code to be generated step-by-step throughout implementation, this allowed for debugging to become more efficient, as ChatGPT had view of everything I was doing at each stage. The final fix still required manual verification and code changes.

Figure A7. Comparative dataset evaluation to justify why the chosen Kaggle source was the strongest fit.



Could you give me other suggestions for datasets, and explain why this one is the most suitable?

Here are **several Kaggle dataset options** that could work for your **Travel Without Barriers API**, followed by an explanation of **why one stands out as the most suitable for your coursework requirements.**

1. Best Overall Dataset (Recommended)

Tourist Destinations Dataset

Dataset: *Popular Tourist Destinations and Their Features*
URL: https://www.kaggle.com/datasets/cosmox23/popular-tourist-destinations-and-their-features

Why it works extremely well

This dataset typically includes fields such as:

- destination name
- country / continent
- destination type (beach, city, nature, etc.)
- **average cost estimate**
- **visitor rating**
- **annual visitor numbers**
- **best season**

These fields map **almost perfectly** onto the recommendation goals of your API.

Your API wants to recommend destinations that are:

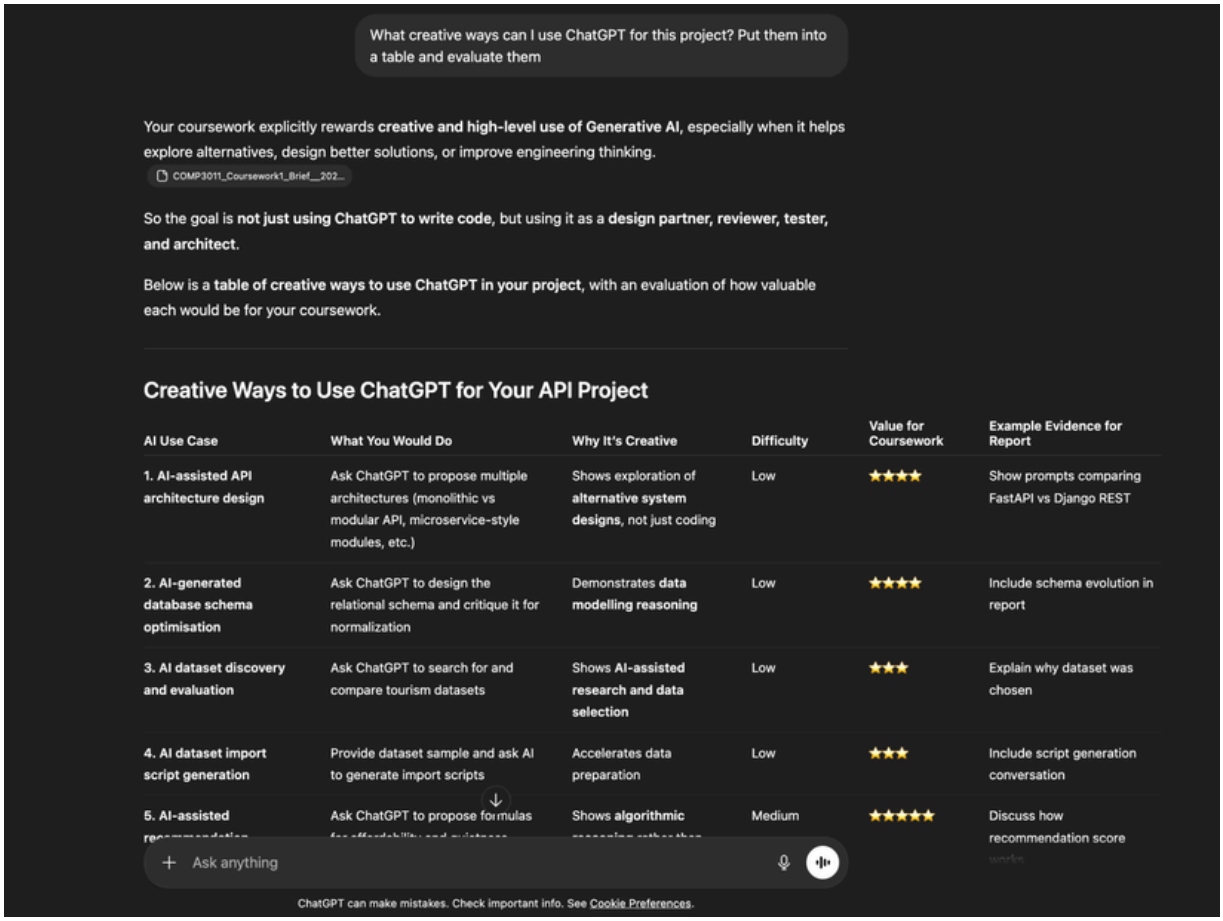
Travel Without Barriers Goal	Dataset Feature
Affordable travel	avg_cost_usd
Less crowded	annual_visitors
Highly rated	rating

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

This conversation provided a second layer of justification for dataset selection (shown in A2) by comparing alternatives and explaining why the chosen dataset mapped best to the project goal. This helped transform dataset selection from a casual choice into a defensible design decision.

Figure A8. Brainstorming creative ways to use ChatGPT within the coursework itself.



What creative ways can I use ChatGPT for this project? Put them into a table and evaluate them

Your coursework explicitly rewards **creative and high-level use of Generative AI**, especially when it helps explore alternatives, design better solutions, or improve engineering thinking.

COMP3011_Coursework1_Brief__202...

So the goal is **not just using ChatGPT to write code**, but using it as a **design partner, reviewer, tester, and architect.**

Below is a **table of creative ways to use ChatGPT in your project**, with an evaluation of how valuable each would be for your coursework.

Creative Ways to Use ChatGPT for Your API Project

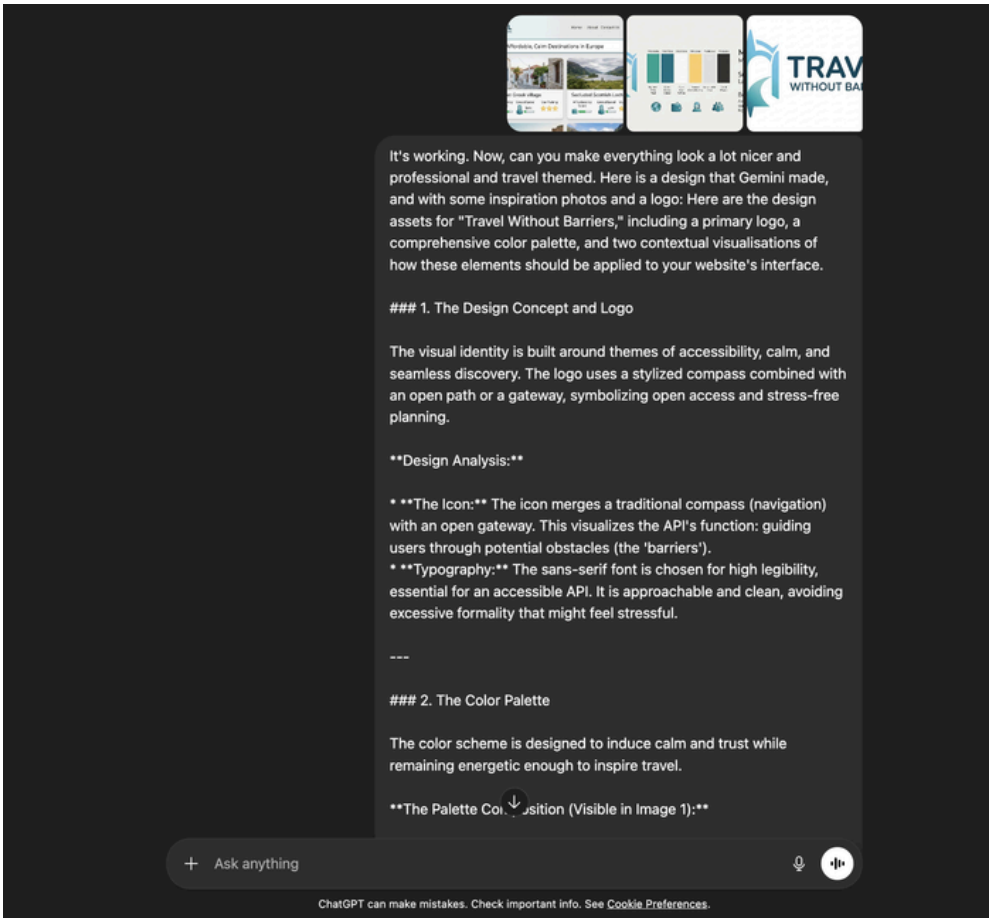
AI Use Case	What You Would Do	Why It's Creative	Difficulty	Value for Coursework	Example Evidence for Report
1. AI-assisted API architecture design	Ask ChatGPT to propose multiple architectures (monolithic vs modular API, microservice-style modules, etc.)	Shows exploration of **alternative system designs**, not just coding	Low	★★★★	Show prompts comparing FastAPI vs Django REST
2. AI-generated database schema optimisation	Ask ChatGPT to design the relational schema and critique it for normalization	Demonstrates **data modelling reasoning**	Low	★★★★	Include schema evolution in report
3. AI dataset discovery and evaluation	Ask ChatGPT to search for and compare tourism datasets	Shows **AI-assisted research and data selection**	Low	★★★	Explain why dataset was chosen
4. AI dataset import script generation	Provide dataset sample and ask AI to generate import scripts	Accelerates data preparation	Low	★★★	Include script generation conversation
5. AI-assisted	Ask ChatGPT to propose formulas	Shows **algorithmic**	Medium	★★★★★	Discuss how recommendation score

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

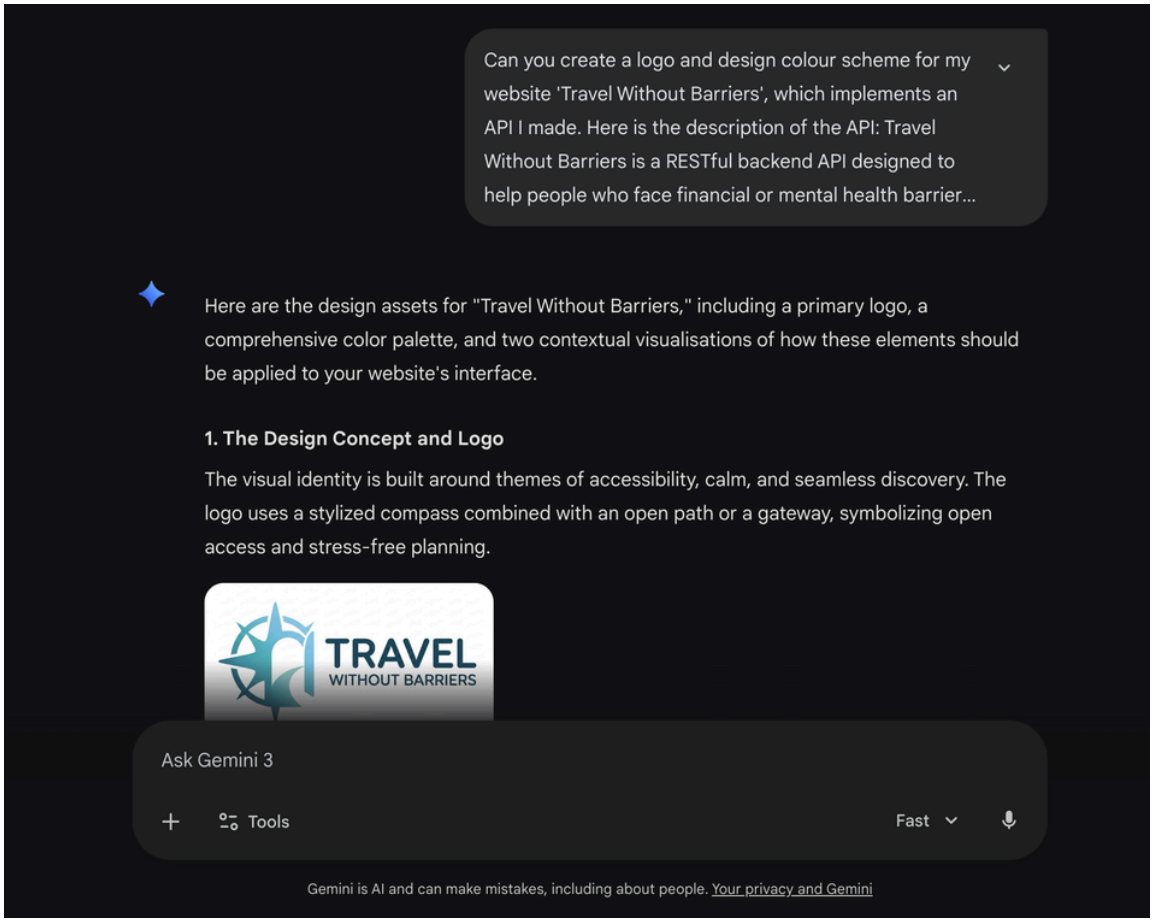
This screenshot shows reflective use of AI at the meta level: not only asking for outputs, but explicitly evaluating how AI could be used meaningfully within the project. This supported the later decision to include MCP, prompt engineering, debugging, dataset selection, and documentation assistance as deliberate AI-assisted workflows.

Figure A9. Gemini used to create brand and interface design guidance for Travel Without Barriers.



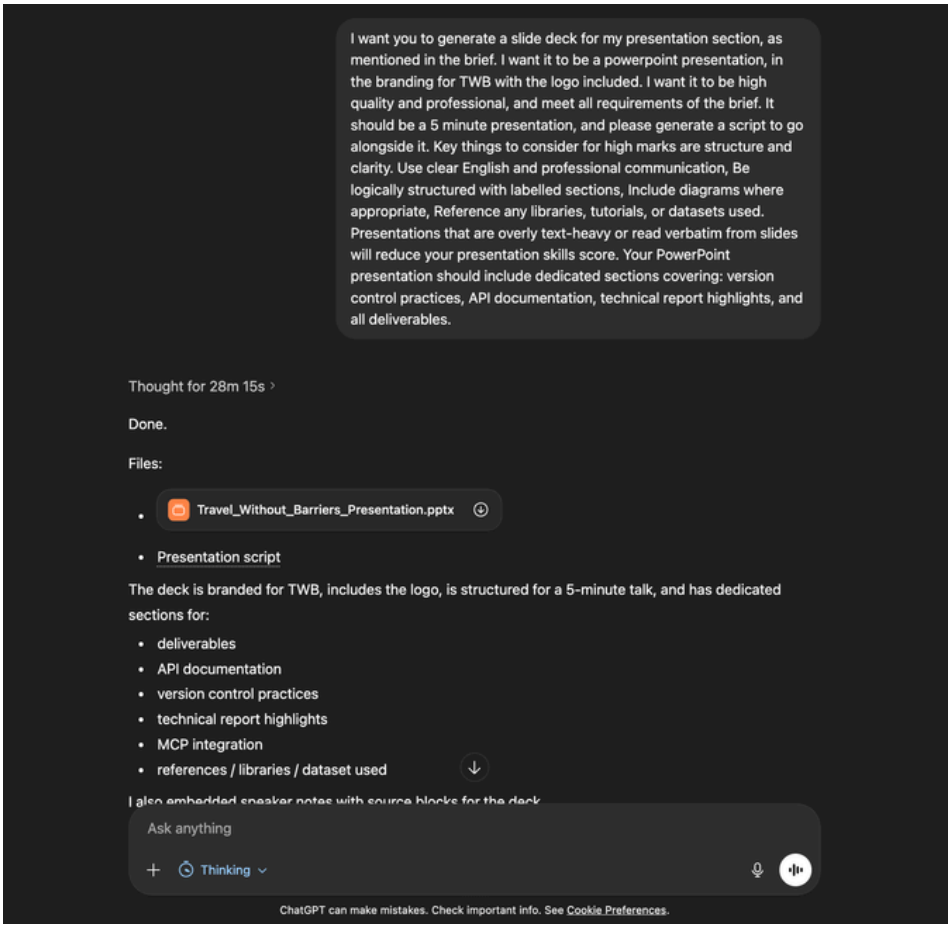
This shows me providing the branding generated by Gemini to ChatGPT and using it to design the frontend of the project. This screenshot shows Gemini being used as a secondary tool, specifically for visual identity and interface styling. The value here was comparative and creative rather than purely technical: it helped shape branding decisions, travel-themed presentation, and a calmer visual tone consistent with the product goal. The prompt in this screenshot was created using GenAI.

Figure A10. Gemini-generated branding assets, including the logo and colour direction used in the project.



This screenshot is connected to Figure A9 and evidences the use of a second GenAI tool (Gemini) to generate the project's visual brand. The generated logo and palette informed the visual identity used in the frontend and documentation, showing that GenAI supported both engineering and communication outcomes.

Figure A11. GenAI used to create Powerpoint presentation and script



This screenshot shows ChatGPT being used to create a Powerpoint presentation as part of the coursework brief. ChatGPT created the design, which were then reviewed by me and adapted where necessary. ChatGPT also created a draft script to use alongside the slides.

Appendix B — Dataset Licence Notice

The dataset used in this project was sourced from Kaggle: Popular Tourist Destinations and Their Features. The following licence text was provided by the user for inclusion in the report and is reproduced here to document academic use and attribution expectations.

Released under MIT License

Copyright (c) 2013 Mark Otto.

Copyright (c) 2017 Andrew Fong.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.